

Bug Hunt v3: E-Commerce Order Pipeline

This is a **group** bug hunt for six to eight students. Plan on at least an hour, and divide the reading among your team — no one person can carefully read every module in the time available.

What the program does

This program models the order-processing pipeline of a small online store. It is spread across **eight modules**:

- `product.erl` — the product ADT (sku, name, price, weight, category).
- `customer.erl` — the customer ADT (id, name, email, loyalty tier).
- `inventory.erl` — multi-warehouse stock. Tracks on-hand and reserved quantities per (warehouse, sku). Implements multi-warehouse reservation.
- `pricing.erl` — tax rates by category, shipping by weight, promo-code lookup, loyalty-tier discount rates.
- `cart.erl` — the shopping-cart ADT (line items plus a promo code, linked to a customer record). Computes subtotal, tax, shipping, promo discount, tier discount, and total.
- `order.erl` — the order ADT and its state machine: `pending` → `paid` → `shipped`, plus `cancel/1`.
- `fulfillment.erl` — drives orders through the pipeline: `pay`, `reserve stock`, `ship`. Handles per-order rollback when stock is insufficient.
- `main.erl` — the driver. Builds the scenario, runs the pipeline, prints the report.

Intended behavior

- **Product accessors** return the field they are named after. `weight_g/1` returns weight in grams; `price_cents/1` returns price in cents; etc.
- **Customer accessors** return the field they are named after. `tier/1` returns the loyalty-tier atom (`bronze`, `silver`, `gold`, or `platinum`).
- **Tax rates** by category, in basis points (1 bps = 0.01%): apparel 800, electronics 950, books 0, supplies 600, everything else 0. Tax is per line: `unit_price * qty * rate_bps div 10000`, summed across lines.
- **Shipping cost** by total cart weight (grams), inclusive on the upper end:

Programming Language Fundamentals

Oregon State University Cascades

Spring 2026

Weight (g)	Shipping
0 to 500	\$4.99
501 to 1500	\$7.99
1501 to 5000	\$12.99
5001 and up	\$19.99

A 500-gram order ships at \$4.99, *not* \$7.99.

- **Promo codes** are case-sensitive strings. "SAVE10" is 1000 bps (10%), "VIP20" is 2000 bps (20%), "" or an unknown code yields 0%.
- **Loyalty tier discounts:** bronze 0%, silver 2%, gold 5%, platinum 10%. Anything else (including a malformed tier value) yields 0%.
- **Discount semantics:** both the promo discount and the tier discount are computed as a percentage of the *subtotal only* — not of tax, not of shipping. The two discounts *stack*. The total is then

$$\text{total} = \text{subtotal} - \text{promo_discount} - \text{tier_discount} + \text{tax} + \text{shipping}$$

- **Inventory** is a list of warehouses, in the order they were added (**north**, then **south**, then **east**). Each warehouse holds per-sku triples {**Sku**, **OnHand**, **Reserved**}. The conservation invariant: for every sku, the sum of (**OnHand** + **Reserved**) across all warehouses stays constant from initial stocking through the program's run. Units only move between buckets, never created or destroyed.
- **Multi-warehouse reservation** walks warehouses in current list order. The first warehouse with any stock takes what it can (up to the amount needed); if more is still needed, the next warehouse with stock takes more; and so on. The returned warehouse list **must be in the same order as the input** — only the per-sku (**OnHand**, **Reserved**) numbers change.
- **Order state machine:** pending → paid → shipped, plus a **cancel** transition. **cancel** is allowed from **pending** or **paid** only. **shipped** and **cancelled** are terminal — an attempt to cancel a shipped order must return {**error**, {**cannot_cancel**, **shipped**}} and leave the order untouched.
- **Fulfillment** runs orders in input order. For each order it marks the order **paid**, attempts to reserve every line item, and marks the order **shipped** with the allocations on success. If *any* line in an order cannot be fully satisfied, the order is rolled back: it stays in **paid**, no allocations are recorded, and the inventory is restored to its pre-order state. Crucially, fulfillment then continues with the next order — one failed order must not stop the rest of the queue.

The scenario

- **Catalog:** 5 products, shown in the report.
- **Customers:** 5 customers — alice (silver), bob (bronze), carol (gold), dave (platinum), eve (bronze).
- **Inventory:** 3 warehouses, stocked as shown.

Programming Language Fundamentals

Oregon State University Cascades

Spring 2026

- **Carts:** each customer has a cart with line items and an optional promo code.
- **Pipeline:** each cart becomes an order; fulfillment runs over the whole list.
- One of the five orders (dave's) cannot be fulfilled because it asks for more gadgets than the entire chain has on hand. That order remains in **paid**. Eve's order, which comes after dave's, should still ship normally.
- After fulfillment, the driver attempts to cancel order #2 (bob's), which has already shipped. This attempt must fail.

The catch

This program contains **between 10 and 18 bugs**. The bugs are all *semantic* — the program compiles without errors (no warnings) and runs to completion without crashing.

This is a v3 problem. Six specific traps to watch for:

1. **You don't know the exact bug count.** A few “fixes” you might propose are not fixes at all — they only look like they should be, because pieces of the program are conspiring to produce plausible-looking wrong output. Verify every fix against the expected output below.
2. **Bugs are spread across the file tree.** No module is bug-free, and the symptom of a bug often appears in a module that is not the bug's cause. Plan to divide reading by module among your group.
3. **At least one bug hides another, and at least one bug hides *two* others.** Fixing the louder one will change which output lines are wrong. Don't assume your other fixes are broken when this happens — assume there's another bug to find.
4. **Cross-group dynamics are real.** When the group that owns one module fixes a bug, the symptoms that show up in a different group's output may *change* or even get worse before they get better. A team leader is the one who keeps the picture coherent — when group X says “I fixed mine and it looks the same” and group Y says “I think your fix broke something for me,” they're usually both right, and there's a third bug to find that makes both pictures snap into alignment.
5. **At least one bug is conservation-preserving.** A “did the total stay the same?” sanity check will not detect it. The report prints per-warehouse on-hand and reserved counts plus a total-stock summary; you'll need to compare those line by line.
6. **At least one bug is a structural / cross-file bug:** the value travels from one module through several others before it produces a wrong number. Trace by following data, not by reading files top-to-bottom.

You may not run the program until you have submitted your answer. Trace by reading.

Programming Language Fundamentals

Oregon State University Cascades

Spring 2026

Expected output

The complete expected output for the all-bugs-fixed run is in the file `expected_output.txt` that accompanies this problem set. Compare your buggy program's output to it line by line.

A few specific things to look for:

- Warehouses always appear in the order **north**, **south**, **east** — the order they were added.
- The customer line of each order's pricing section shows the tier atom (**silver**, **bronze**, **gold**, **platinum**) — *not* an email string. If you see something like "alice@shop.com" there, a bug in `customer.erl` is leaking.
- Alice gets *both* the 10% promo and the 2% silver tier discount. Carol gets only the 5% gold tier discount (no promo). Dave gets both the 20% promo and the 10% platinum tier discount.
- Order #4 (dave) is the only fulfillment failure. It stays **paid** with no allocations. Order #5 (eve) shipping *after* #4 demonstrates that fulfillment continues past a failed order.
- Order #2 (bob) ships normally; the cancellation attempt at the end is rejected.
- Total stock per sku is unchanged from the initial inventory. Revenue from shipped orders is \$618.57.

Your write-up

For each bug, record:

1. **Location** — file name and line number(s).
2. **Diagnosis** — in 1–2 sentences, what is wrong.
3. **Fix** — the corrected line(s).
4. **Symptom** — which line(s) of the expected output differ from what the buggy program actually prints, and why this specific bug causes that specific symptom. If a bug has no direct visible symptom (because another bug masks it), explain how you knew it was there anyway.
5. **Bonus**: for any bug that hides or is hidden by another, name the relationship and which group originally spotted each one. Multi-group hiding chains (one fix in group X reveals a bug in group Y) are worth extra credit.